

高级语言程序设计 实验报告

南开大学 计算机大类

方俊哲

2511142

0572

2026 年 5 月 8 日

目录

- 一、作业题目 1
- 二、开发软件 1
- 三、课题要求 2
 - 1. 课程要求 2
 - 2. 项目目标 2
 - 3. 项目范围 3
- 四、主要流程 3
 - 1. 整体流程 3
 - 2. 功能模块设计 5
 - (1) Data 模块 5
 - (2) Logic 模块 6
 - (3) Interaction 模块 6
 - (4) assets 模块 7
 - (5) config 模块 8
 - 3. 核心类与数据结构 8
 - (1) GameManager 类 8
 - (2) Store 类 9
 - (3) Product 类 10
 - (4) Employee 类 10
 - (5) 页面类 11
 - (6) 核心交互机制详解 11
 - 4. 主要算法与规则 12
 - (1) 商品采购规则 12
 - (2) 商品价格调整规则 12
 - (3) 简化营业结算模型 13
 - (4) 营业结算规则 13

(5) 店铺升级判断规则 13

(6) 轻量化员工展示规则 14

(7) 存档读写规则 14

(8) 资源加载容错规则 14

(9) 页面切换与流程控制规则 15

(10) 交互优化规则 15

五、功能测试 16

1. 测试目的 16

2. 测试环境 17

3. 边界测试与异常测试 17

4. 测试结果分析 18

六、版本迭代与项目管理 18

七、AI 工具使用说明 19

八、收获 20

一、作业题目

本次高级语言程序设计大作业的项目名称为 Tideshore / 潮汐小镇。

本项目是一个基于 C++ 与 Qt 开发的 2D 像素风海边便利店模拟经营游戏。程序采用 Qt6 / Qt Widgets 实现图形化界面，以海边便利店经营为主题，围绕商品采购、商品定价、每日事件、营业模拟、每日结算、店铺升级等内容构建基本的经营流程。

项目的主要目标不是实现复杂的角色移动或大型游戏系统，而是在课程大作业范围内，完成一个具有完整流程、基本交互和可运行效果的 C++ 图形化小程序。程序主流程已经包括启动页、营业前准备、营业中、每日结算、店铺升级以及进入下一天等环节，能够形成相对完整的多日经营循环。

二、开发软件

本项目的开发环境和相关工具如下：

项目	内容
操作系统	Windows
编程语言	C++
图形界面框架	Qt6 / Qt Widgets
集成开发环境	CLion
构建工具	CMake、Ninja
编译器	MinGW g++
版本管理工具	Git、GitHub
项目资源目录	assets
配置文件目录	config
主要代码结构	Data、Logic、Interaction

项目使用 C++ 作为主要开发语言，通过 Qt6 / Qt Widgets 实现图形化窗口、页面切换、按钮交互、界面显示和用户操作反馈。项目采用 CMake 组织工程构建，并使用 Ninja 作为构建工具，配合 MinGW g++ 完成编译运行。

在代码结构方面，项目按照功能职责进行了基本划分，主要包括 Data、Logic、Interaction、assets、config 等部分。其中，Data 主要用于组织数据相关内容，Logic 主要用于实现经营规则和流程控制，Interaction 主要用于实现界面交互和页面逻辑，assets

用于保存图像、音效等资源文件，config 用于保存配置相关内容。通过这种划分，可以避免将所有功能集中写在单一文件中，使项目结构更清晰，也便于后续调试与维护。

项目开发过程中使用 Git 进行版本管理，并通过 GitHub 保存项目代码和版本记录。Release 打包已经完成，可以用于课程展示和程序运行测试。

三、课题要求

1. 课程要求

本次高级语言程序设计大作业要求使用 C++ 完成一个图形化小程序。根据课程要求，项目需要体现 C++ 程序设计的基本能力，包括类与对象的使用、数据组织、函数封装、模块划分、程序流程控制以及基本的调试与测试能力。

本项目选择 Qt 作为图形界面开发框架，使用 Qt Widgets 构建程序窗口和多个功能页面。相较于普通控制台程序，图形化程序需要处理更多用户交互、页面切换、按钮响应和界面刷新问题，因此能够较好地体现 C++ 在实际项目中的综合应用。

在本项目中，程序已经实现启动页、营业前准备、营业中、每日结算、店铺升级和进入下一天等主要页面流程。商品采购、商品价格调整、每日事件显示、资金和库存更新、每日结算展示、店铺升级、存档读档、UI 音效反馈和鼠标点击特效等功能均已完成，能够满足课程大作业中“完成一个可运行图形化小程序”的基本要求。

2. 项目目标

本项目的目标是在有限开发周期内，完成一个结构较清晰、功能较完整、能够实际运行和演示的 C++ / Qt 图形化程序。项目以海边便利店经营为主题，通过“营业前准备—营业中—每日结算—店铺升级—进入下一天”的流程，构建一个基本的模拟经营循环。

具体而言，项目希望实现以下目标：

首先，完成基本的图形化界面。程序使用 Qt6 / Qt Widgets 实现多个页面，并通过页面切换展示不同阶段的经营内容，使用户能够通过按钮和界面控件完成主要操作。

其次，完成基础经营逻辑。玩家可以在营业前进行商品采购和商品价格调整，程序根据资金和库存状态更新相关数据；每日事件能够在界面中显示，并作为经营过程中的信息提示；营业结束后，程序可以进入每日结算页面，展示经营结果。

再次，完成长期成长与状态保存。项目已经实现店铺升级功能，玩家可以将经营获得的资金投入后续发展。同时，存档系统已经完全实现，能够保存和恢复游戏进度，使程序具备连续游玩的基本条件。

最后，项目还完成了统一 UI 音效反馈和鼠标点击特效，使用户操作具有更明确的反馈。这些内容主要用于改善程序交互体验，但不改变项目的核心定位。

3. 项目范围

本项目的范围控制在课程大作业适合完成的规模内，重点实现基础经营流程和图形化交互，不追求复杂大型游戏系统。

项目已经完成的主要范围包括：启动页、营业前准备页面、营业中页面、每日结算页面、店铺升级页面、进入下一天流程、商品采购、商品价格调整、每日事件显示、资金和库存更新、每日结算展示、店铺升级、轻量化员工展示、存档系统、统一 UI 音效反馈、鼠标点击特效以及 Release 打包。

其中，轻量化员工展示主要包括雇佣状态和角色展示，用于增强店铺经营的表现效果。该部分不属于复杂员工管理功能，不涉及复杂排班、工资博弈、情绪管理等机制。

本项目没有将复杂角色自由移动、复杂顾客寻路、大地图探索、多楼层店铺、复杂剧情系统等内容作为主要实现目标。对于未实现或未确认的功能，报告后续部分将不进行扩展描述；如确需提及，将统一标注为“待确认”。

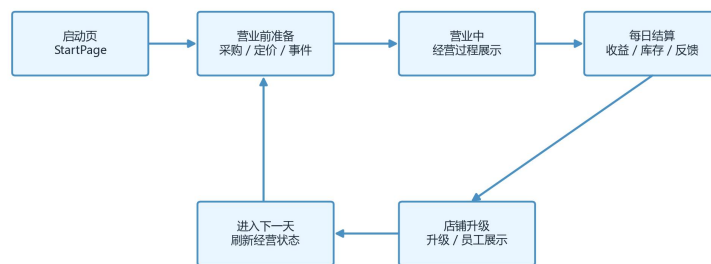
四、主要流程

1. 整体流程

本项目采用循环式模拟经营流程。玩家以“天”为基本单位推进游戏，每一天都按照相对固定的流程进行：先在营业前完成经营决策，再进入营业中观察经营结果，随后通过每日结算查看本日经营情况，最后在店铺升级阶段进行长期发展，之后进入下一天继续经营。

项目当前已经完成的主流程如下：

图1 Tideshore / 潮汐小镇整体经营流程



主循环：营业前决策 → 营业过程 → 结算反馈 → 长期成长 → 下一天

首先，程序启动后进入启动页。启动页主要用于展示项目入口，并提供开始游戏等基础操作。玩家从启动页进入游戏后，程序开始初始化当前经营状态，包括天数、资金、商品、库存等基本数据。

进入营业前准备阶段后，玩家可以查看当前资金、商品信息、库存情况和每日事件，并进行商品采购和商品价格调整。该阶段是玩家进行主要经营决策的阶段，玩家需要根据已有资金、商品进价、库存容量和事件提示决定当天的采购数量与销售价格。项目已经实现商品采购、商品价格调整、每日事件显示，以及资金和库存数据的更新。

营业前准备完成后，玩家进入营业中阶段。该阶段用于表现当天经营过程，程序根据已经设定好的商品库存、商品售价和当日经营状态推进营业过程。营业中阶段不是复杂实时操作系统，而是以经营结果展示和轻量交互为主，使玩家能够看到前一阶段经营决策带来的影响。同时，项目已经完成统一 UI 音效反馈和鼠标点击特效，使按钮点击和界面操作具有较明确的反馈。

营业结束后，程序进入每日结算阶段。每日结算页用于展示当天经营结果，包括营业情况、资金变化和库存变化等内容。通过结算页面，玩家可以了解当天经营是否有效，并为下一天的进货和定价提供参考。项目当前已经完成每日结算展示功能。

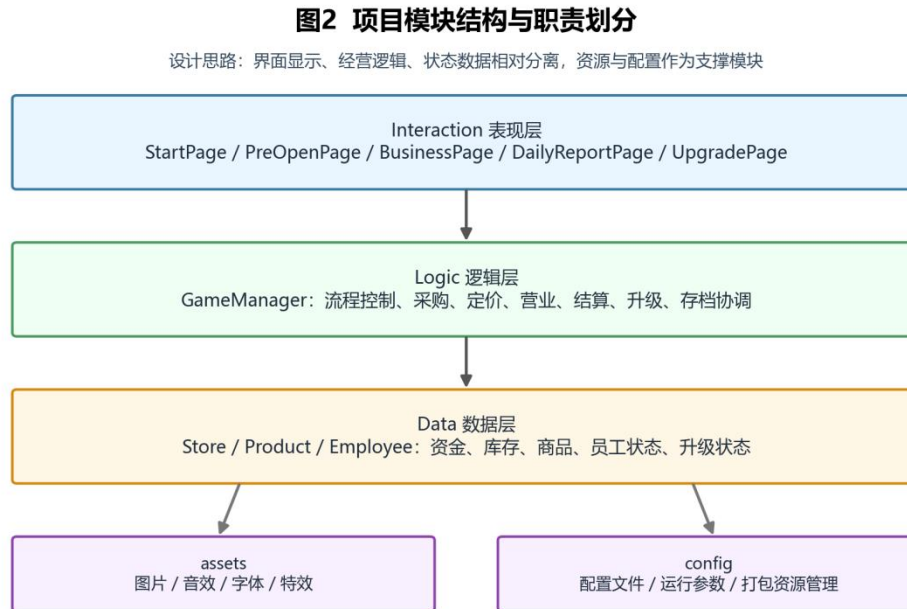
每日结算之后，玩家进入店铺升级阶段。该阶段用于长期成长，玩家可以使用经营获得的资金进行店铺升级，也可以查看或进行轻量化员工雇佣状态与形象展示。该部分只保留较简单的雇佣状态和角色展示，不涉及复杂排班、工资博弈或情绪管理等机制。

完成升级操作后，玩家可以选择进入下一天。程序进入下一轮营业前准备阶段，刷新或更新对应经营数据，从而形成“准备—营业—结算—升级—下一天”的循环式模拟经营流程。该流程既保证了程序结构清楚，也使项目具有连续运行和多日经营的基本条件。

2. 功能模块设计

本项目的实际代码结构主要包括 Data、Logic、Interaction、assets、config 五个部分。各部分按照职责进行划分，使数据、逻辑、界面交互和资源文件相对分离，避免所有功能集中在单一文件或单一模块中。

项目模块职责如下表所示：



(1) Data 模块

Data 模块主要负责项目中的数据组织和状态保存。对于模拟经营类程序而言，数据是经营流程的基础，商品信息、资金状态、库存数量、员工状态、每日事件和存档信息都需要通过合理的数据结构进行保存。

在本项目中，Data 模块主要服务于以下内容：

- 商品数据
- 员工数据
- 事件数据
- 资金与库存状态
- 每日经营结果
- 存档相关数据

商品数据用于记录商品名称、价格、库存等信息，是商品采购、商品价格调整和每日结算的基础。资金与库存状态用于反映玩家当前经营情况，在采购、营业、结算和升级过程中会不断发生变化。员工数据主要用于轻量化雇佣展示，记录员工是否已被雇佣以及相关展示状态。存档相关数据用于保存和恢复游戏进度，使玩家能够在关闭程序后继续之前的经营状态。

通过将这些内容集中放在 Data 模块中，可以使程序中的基础数据结构更加清晰，也便于其他模块读取和修改。

(2) Logic 模块

Logic 模块主要负责项目中的核心经营规则和流程控制，是整个项目的主要逻辑部分。该模块不直接负责界面显示，而是处理用户操作后对应的数据变化和经营结果。

本项目中，Logic 模块主要包括以下功能：

- 商品采购逻辑
- 商品价格调整逻辑
- 资金更新逻辑
- 库存更新逻辑
- 每日事件处理逻辑
- 每日结算逻辑
- 店铺升级逻辑
- 存档读写逻辑
- 进入下一天的流程控制

在商品采购过程中，程序需要根据玩家选择的商品和采购数量，判断资金是否足够，并更新资金与库存。商品价格调整功能则用于修改商品的当前销售价格，为营业和结算提供依据。

每日事件处理逻辑用于显示和管理当天事件，使经营过程具有一定变化。每日结算逻辑根据当天经营情况统计结果，并将结果传递给结算页面进行展示。店铺升级逻辑用于处理升级操作，包括判断资金是否满足要求、扣除资金并更新店铺状态。

此外，项目已经完全实现存档系统，因此 Logic 模块还承担保存和恢复游戏进度的功能。存档系统需要记录当前天数、资金、商品库存、升级状态、员工雇佣状态等内容，使程序重新打开后能够恢复之前的游戏状态。

(3) Interaction 模块

Interaction 模块主要负责 Qt 图形化界面和用户交互，是玩家直接看到和操作的部分。本项目使用 Qt6 / Qt Widgets 实现图形化界面，因此页面、按钮、提示信息、界面刷新和页面切换都属于该模块的重要内容。

项目当前已经完成的主要页面流程包括：

- 启动页
- 营业前准备页
- 营业中页
- 每日结算页
- 店铺升级页
- 进入下一天流程

启动页作为程序入口，负责进入游戏主流程。营业前准备页负责展示商品、资金、库存、每日事件等信息，并接收玩家的采购和价格调整操作。营业中页负责展示当天营业过程和相关交互反馈。每日结算页负责展示当天经营结果。店铺升级页负责展示升级内容和轻量化员工雇佣状态。

Interaction 模块通过按钮点击、页面切换和界面刷新与 Logic 模块发生联系。例如，玩家在营业前准备页点击采购按钮后，界面层会将操作传递给逻辑层，逻辑层完成资金和库存更新后，再由界面层显示新的数据。这样可以使界面显示和业务逻辑保持一定程度的分离。

(4) assets 模块

assets 模块用于保存项目运行所需的资源文件。由于本项目是 2D 像素风图形化程序，界面表现需要使用一定数量的图片、图标、音效和字体资源。

本项目中，assets 模块主要包括以下类型资源：

- 页面背景资源
- 按钮与 UI 图标资源
- 商品或角色展示资源
- 员工角色展示资源
- 音效资源
- 鼠标点击特效资源
- 字体资源

其中，图像资源用于构建游戏页面的视觉效果；按钮和图标资源用于增强界面可读性；员工角色资源用于轻量化员工雇佣状态与形象展示；音效资源用于统一 UI 音效反馈；鼠标点击特效资源用于增强操作反馈。

资源文件与代码文件分离，有利于后续替换素材和调整界面风格，也可以避免将资源路径和界面逻辑混杂在一起。

(5) config 模块

config 模块主要用于保存项目配置相关内容。该模块的作用是将部分可配置内容从程序逻辑中分离出来，便于管理和修改。

在本项目中，config 模块可以用于组织资源路径、基础参数或其他运行配置。对于需要在程序运行中读取的固定配置内容，将其放入 config 中可以提高项目结构的清晰度。

在整体结构中，config 模块不直接承担经营计算或界面显示功能，而是为 Data、Logic 和 Interaction 等模块提供必要的配置支持。通过保留独立的配置目录，可以使项目后续维护更加方便，也便于在 Release 打包时统一管理运行所需文件。

3. 核心类与数据结构

本项目采用 C++ 面向对象的方式组织主要程序内容。由于项目包含经营数据、业务逻辑和图形界面交互等不同部分，因此在实现时将主要职责分散到不同类中，避免将所有代码集中在单一页面或单一函数内。

项目中的核心类主要可以分为两类：一类是保存游戏状态和经营数据的类，如 GameManager、Store、Product、Employee；另一类是负责 Qt 页面显示和用户交互的页面类，如 StartPage、PreOpenPage、BusinessPage、DailyReportPage、UpgradePage 等。

核心类职责如下表所示：

类名	所属部分	主要职责
GameManager	Logic	管理游戏整体流程，协调每日开始、营业、结算、升级、进入下一天等操作
Store	Data	保存店铺当前状态，包括资金、库存、等级、员工雇佣状态等
Product	Data	保存单个商品的基本信息和经营状态，如名称、价格、库存等
Employee	Data	保存员工的基础信息和雇佣状态，用于轻量化员工展示
StartPage	Interaction	实现启动页面，作为程序入口
PreOpenPage	Interaction	实现营业前准备页面，处理商品采购、价格调整、事件显示等交互
BusinessPage	Interaction	实现营业中页面，展示营业过程和相关操作反馈
DailyReportPage	Interaction	实现每日结算页面，展示当日经营结果
UpgradePage	Interaction	实现店铺升级页面和轻量化员工雇佣展示
存档相关类或函数	Logic / Data	负责游戏进度的保存和恢复
资源加载相关函数	Interaction / assets	负责图片、音效等资源加载，并在资源缺失时进行容错处理

(1) GameManager 类

GameManager 是项目中负责整体流程控制的核心类，可以理解为游戏运行过程中的总控制器。它不直接承担界面绘制工作，而是负责协调经营流程中的关键状态变化。

在本项目中，GameManager 主要负责以下内容：

- 开始新游戏
- 进入新的一天

- 处理商品采购
- 处理商品价格调整
- 开始营业
- 结束营业并生成结算结果
- 处理店铺升级
- 处理员工雇佣状态
- 保存和恢复游戏进度

例如，当玩家在营业前准备页面完成商品采购和价格调整后，程序会通过相应的逻辑进入营业中阶段；营业结束后，再由 GameManagerer 协调生成每日结算数据，并切换到每日结算页面。进入下一天时，GameManagerer 需要更新天数、经营状态和页面显示数据，从而保证整个游戏流程能够循环进行。

通过设置 GameManagerer，项目可以将“游戏流程控制”与“界面显示”分离，避免每个页面都直接修改大量全局状态，使程序结构更加清楚。

(2) Store 类

Store 类用于保存店铺当前的整体经营状态，是项目中的重要数据类。模拟经营类程序需要频繁读取和修改资金、库存、店铺等级等信息，因此将这些内容集中保存在 Store 中，有利于统一管理。

Store 类主要保存的信息包括：

- 当前资金
- 当前天数
- 店铺等级
- 库存状态
- 商品列表
- 员工雇佣状态
- 升级状态
- 其他与店铺经营相关的运行状态

在商品采购时，程序需要读取 Store 中的当前资金和库存信息，并在采购成功后更新资金与库存。在每日结算时，程序会根据商品销售结果更新资金和库存。在店铺升级时，程序也需要判断当前资金是否满足升级条件，并在升级成功后更新对应店铺状态。

因此，Store 类的主要作用是保存“当前店铺是什么状态”，而具体的计算和判断逻辑则由其他逻辑代码完成。

(3) Product 类

Product 类用于表示单个商品。商品是本项目经营系统的基础对象，玩家的采购、定价、库存变化和每日结算都围绕商品展开。

Product 类中通常需要保存以下信息：

- 商品名称
- 商品类别
- 商品进价
- 商品售价
- 当前库存
- 是否已解锁
- 其他与商品经营相关的状态

在营业前准备阶段，玩家可以查看商品信息，并对商品进行采购和价格调整。商品采购会增加对应商品库存，价格调整会改变该商品在营业阶段使用的销售价格。营业结束后，程序根据销售情况减少商品库存，并将经营结果汇总到每日结算页面中。

Product 类主要承担商品数据保存的作用，不宜包含过多复杂界面逻辑。这样可以使商品数据结构更稳定，也方便后续在界面中展示商品信息。

(4) Employee 类

Employee 类用于保存员工相关信息。本项目中的员工雇佣状态与形象展示采用轻量化实现，主要包括员工是否已雇佣以及对应角色展示，不涉及复杂员工排班、工资博弈、情绪管理等机制。

Employee 类主要保存的信息包括：

- 员工名称
- 员工类型或角色
- 是否已雇佣
- 角色展示资源
- 与雇佣展示相关的基础状态

在店铺升级页面中，玩家可以查看员工信息并进行雇佣操作。雇佣后，程序更新员工状态，并在界面中显示对应角色或状态变化。该部分的主要作用是增强项目表现效果和经营成长感，而不是构成复杂的人力资源管理功能。

(5) 页面类

本项目使用 Qt6 / Qt Widgets 实现图形化界面，主要页面类包括 StartPage、PreOpenPage、BusinessPage、DailyReportPage、UpgradePage 等。这些页面类主要负责界面显示、用户输入和按钮交互。

页面类职责如下表所示：

页面类	主要功能	对应流程阶段
StartPage	显示项目启动页，提供进入游戏的入口	启动阶段
PreOpenPage	显示资金、商品、库存、事件信息，支持采购和价格调整	营业前准备
BusinessPage	展示营业中状态，配合 UI 音效和点击特效提供交互反馈	营业中
DailyReportPage	展示每日结算结果，包括经营情况和资金变化等	每日结算
UpgradePage	展示店铺升级内容和轻量化员工雇佣状态	店铺升级
页面切换控制	根据游戏流程在不同页面之间切换	全流程

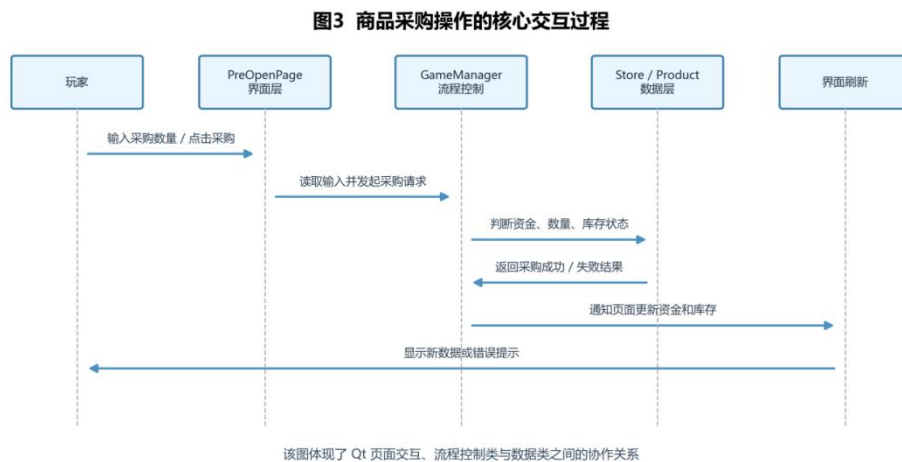
其中，PreOpenPage 是玩家进行主要经营决策的页面。玩家在该页面中完成商品采购和商品价格调整，相关操作会影响后续营业和结算结果。BusinessPage 用于承接营业阶段，展示经营过程。DailyReportPage 负责将当天经营结果清晰展示给玩家。UpgradePage 则用于处理店铺升级和员工雇佣状态与形象展示，完成后进入下一天。

页面类主要负责“显示”和“交互”，而不应承担过多核心经营计算。通过将页面类与数据类、逻辑类分离，程序结构更加符合模块化设计思想。

(6) 核心交互机制详解

本项目的图形化交互主要依靠 Qt 的信号槽机制完成。按钮、输入框等控件负责接收玩家操作，页面类负责读取用户输入并触发对应逻辑，GameManager 负责协调流程和数据修改，Store 与 Product 等数据类保存实际状态，最后由页面类刷新界面显示。

以“商品采购”为例，其交互过程可以概括为：



在这个过程中，PreOpenPage 不直接承担完整经营规则计算，而是主要负责界面输入和显示；GameManager 负责处理采购行为是否成立；Store 和 Product 保存采购后发生变化的数据。这样可以使页面交互、逻辑处理和数据更新之间形成相对清晰的关系。

该交互结构的优点是职责分明。当采购逻辑需要调整时，主要修改逻辑层代码；当界面布局需要调整时，主要修改页面类代码；当商品数据需要读取或保存时，则集中围绕数据类进行处理。这种方式比把所有判断和界面刷新都写在按钮槽函数中更便于维护，也更符合课程中面向对象程序设计和模块化设计的要求。

4. 主要算法与规则

本项目的主要算法和规则围绕模拟经营流程展开，重点包括商品采购、商品价格调整、营业结算、升级判断、存档读写和资源加载容错等内容。这些规则共同保证程序能够从营业前准备到营业结算，再到店铺升级和进入下一天，形成完整的经营循环。

(1) 商品采购规则

商品采购发生在营业前准备阶段。玩家根据当前资金、商品价格和库存情况选择采购商品。程序在执行采购操作时，需要进行基本合法性判断，避免出现资金不足或库存异常等问题。

商品采购规则说明如下表所示：

判断内容	规则说明
采购数量判断	采购数量应为合理数值，不能为无效输入
资金判断	若当前资金不足，则采购失败，不应扣除资金
库存判断	采购成功后应正确增加对应商品库存
页面刷新	采购后需要刷新资金、库存等界面信息
数据一致性	资金扣除与库存增加应保持同步

通过上述规则，程序可以保证玩家采购行为对经营状态产生正确影响。若采购失败，程序应保持原有数据不变，避免出现资金扣除但库存未增加，或库存增加但资金未扣除等不一致情况。

(2) 商品价格调整规则

商品价格调整同样发生在营业前准备阶段。玩家可以根据商品信息和每日事件提示调整商品售价。商品售价会影响后续营业阶段的经营结果，因此价格调整是本项目经营决策的一部分。

价格调整规则说明如下表所示：

判断内容	规则说明
商品选择	只能对当前可操作的商品进行价格调整
价格合法性	售价应为合理数值，不能为明显无效输入
数据更新	修改后的售价应保存到对应商品数据中
界面显示	调整完成后页面应显示新的商品售价

本项目没有将价格系统设计为复杂金融模型，而是以课程项目可实现、可理解、可展示为原则，使玩家能够通过价格调整参与经营决策。

（3）简化营业结算模型

当前版本不对单个顾客做独立寻路和复杂行为模拟，也不建立完整的顾客 AI。营业阶段主要采用简化结算方式，根据商品库存、商品售价、每日事件和当前经营状态生成总体经营结果。

这种设计符合本项目的课程大作业定位：一方面保留了商品采购、价格调整和事件影响带来的经营反馈；另一方面避免引入复杂寻路、行为树、个体顾客决策等超出当前项目范围的内容。

本项目中的营业结算可以概括为：

营业结果 $\approx$ 商品库存状态 + 商品售价状态 + 每日事件影响 + 当前经营状态

该表达式只用于说明结算思路，并不表示项目实现了复杂数学模型。实际程序以当前已保存的数据为基础生成经营结果，并在每日结算页面中展示资金变化、库存变化和经营反馈。

（4）营业结算规则

营业结算是项目经营循环中的关键环节。玩家在营业前完成采购和定价后，营业阶段会根据已有商品、库存、价格和事件状态产生经营结果。每日结算页用于展示本日经营情况，并更新资金和库存状态。

营业结算规则说明如下表所示：

内容	规则说明
商品库存	商品售出后应减少对应库存
资金变化	商品销售后应增加当前资金
经营结果	结算页应展示当天经营结果
事件影响	每日事件作为经营信息的一部分参与当天经营表现
页面跳转	营业结束后进入每日结算页面

每日结算页面的作用不仅是展示结果，也为下一天的经营决策提供反馈。玩家可以根据当天资金变化、库存变化和经营结果，判断下一天是否需要调整采购和定价策略。

（5）店铺升级判断规则

店铺升级发生在每日结算之后，是长期经营成长的一部分。玩家可以使用经营积累的资金进行店铺升级。升级操作需要进行资金判断，确保只有在资金满足要求时才能完成升级。

升级判断规则说明如下表所示：

判断内容	规则说明
资金判断	当前资金不足时，升级失败
状态更新	升级成功后应更新店铺状态
重复判断	已完成或不满足条件的升级不应重复异常执行
页面反馈	升级成功或失败后应有明确显示或交互反馈

店铺升级的作用是让玩家将每日经营收益转化为后续经营能力，形成“经营获得资金—使用资金升级—进入下一天继续经营”的循环。

(6) 轻量化员工展示规则

本项目中的员工雇佣状态与形象展示采用轻量化实现，主要用于展示员工雇佣状态和角色形象，不包含复杂员工管理机制。员工雇佣通常与升级页面相关联。

该部分的重点是记录员工是否已被雇佣，并在界面上体现对应状态。项目不涉及复杂排班、工资博弈、员工情绪管理等内容。

(7) 存档读写规则

本项目已经完全实现存档系统，可以保存和恢复游戏进度。存档系统对于模拟经营项目较为重要，因为游戏流程以多日循环为基础，玩家需要在关闭程序后继续之前的经营状态。

存档内容通常包括以下几类：

- 当前天数
- 当前资金
- 商品库存
- 商品售价
- 店铺升级状态
- 员工雇佣状态
- 其他必要的经营状态

存档读写规则说明如下表所示：

内容	规则说明
保存时机	在需要保存游戏进度时写入当前状态
读取时机	继续游戏或恢复进度时读取存档
数据完整性	存档应包含恢复游戏所需的关键状态
异常处理	若存档不存在或读取失败，应避免程序崩溃
页面刷新	存档成功后应同步刷新界面显示

通过存档系统，项目不再只是单次运行演示程序，而是具备了保存和恢复经营进度的能力。

(8) 资源加载容错规则

本项目使用了图像、音效、字体、鼠标点击特效等资源文件，因此资源加载是图形化程序运行中的重要部分。为了保证程序稳定性，资源加载需要具备一定容错能力。

资源加载容错规则说明如下表所示：

内容	规则说明
图片资源	缺失时不应导致程序崩溃，可使用默认显示或跳过
音效资源	缺失或加载失败时不影响主要功能运行
字体资源	字体加载失败时可使用系统默认字体
点击特效资源	特效加载失败时不影响按钮和页面操作
错误提示	可通过调试信息提示资源缺失，便于后续修正

资源加载容错的意义在于提高程序稳定性。由于 Release 打包需要同时包含可执行文件和资源文件，如果某些资源路径错误或文件缺失，程序应尽量保持可运行状态，而不是直接崩溃。

（9）页面切换与流程控制规则

项目主流程已经完成，页面切换遵循固定顺序：

启动页 → 营业前准备 → 营业中 → 每日结算 → 店铺升级 → 进入下一天

页面切换规则如下：

阶段	触发操作	下一阶段
启动页	点击开始游戏	营业前准备
营业前准备	完成采购和定价后开始营业	营业中
营业中	营业结束	每日结算
每日结算	点击进入升级	店铺升级
店铺升级	点击进入下一天	下一天营业前准备

通过固定流程控制，程序能够保持清晰的运行顺序，玩家也能明确每个阶段需要完成的操作。该流程既符合模拟经营游戏的基本逻辑，也便于在课程实验报告中说明程序运行过程。

（10）交互优化规则

本项目已经完成统一 UI 音效反馈和鼠标点击特效。这两项功能主要属于交互优化部分，用于增强用户操作反馈，使玩家在点击按钮或进行操作时能够获得更明确的响应。

需要说明的是，UI 音效和鼠标点击特效不属于核心经营算法，也不改变核心经营逻辑。它们不会直接影响商品采购、资金库存、营业结算、店铺升级或存档数据，只用于改善程序操作体验。

该设计使项目在保持核心逻辑清晰的同时，具备更完整的图形化程序交互表现。

五、功能测试

1. 测试目的

本项目为基于 C++ 与 Qt6 / Qt Widgets 开发的图形化模拟经营程序，因此测试重点放在程序能否稳定运行、主要页面能否正常跳转、核心经营功能是否能够正确执行，以及 Release 打包版本是否能够独立运行。

本次测试以功能测试和运行测试为主，不涉及复杂自动化单元测试。测试内容主要围绕项目已经完成的功能展开，包括启动页、营业前准备、商品采购、商品价格调整、每日事件显示、营业中页面、每日结算、店铺升级、轻量化员工雇佣展示、存档系统、统一 UI 音效反馈、鼠标点击特效和 Release 打包版本运行等内容。

测试目标主要包括以下几个方面：

1. 检查程序是否能够正常启动和进入主流程。
2. 检查各页面之间的跳转是否符合设计流程。
3. 检查商品采购、资金扣除、库存增加等核心经营逻辑是否正常。
4. 检查商品价格调整后能否保存并影响后续经营流程。
5. 检查每日事件、营业过程、每日结算和店铺升级等功能是否能够正常展示和执行。
6. 检查轻量化员工雇佣展示是否能够正确反映状态变化。
7. 检查存档系统是否能够保存和恢复游戏进度。
8. 检查 UI 音效反馈、鼠标点击特效和 Release 打包版本是否能够正常运行。
9. 检查部分边界输入和异常情况下程序是否能够保持基本稳定。

2. 测试环境

本项目测试环境如下表所示：

项目	内容
操作系统	Windows
开发环境	CLion
编程语言	C++
图形界面框架	Qt6 / Qt Widgets
构建工具	CMake、Ninja
编译器	MinGW g++
测试版本	Debug 调试版本、Release 打包版本
测试方式	手动功能测试、运行测试

测试过程中，首先在 CLion 开发环境中通过 CMake 和 Ninja 完成项目构建，检查程序是否能够正常编译和运行；随后对各个页面和功能进行手动操作测试；最后对 Release 打包版本进行运行测试，检查程序在打包后是否仍然能够正常启动、加载资源并完成主要流程。

3. 边界测试与异常测试

除常规功能测试外，本项目还对部分边界输入和异常情况进行了检查。由于本项目以课程大作业和手动功能测试为主，该部分测试主要用于确认程序在常见异常输入或资源缺失情况下不会出现明显崩溃或严重数据错误。

边界测试与异常测试如下表所示：

表：边界测试与异常测试补充

编号	测试场景	输入 / 操作	预期结果
B01	采购数量为 0	输入 0	不执行采购，资金和库存不变
B02	采购数量为负数	输入 -5	拒绝输入或提示错误
B03	采购数量超大	输入 999999	资金不足或超过限制，不修改状态
B04	售价为负数	输入 -10	拒绝非法售价
B05	资金刚好等于总价	资金 = 采购总价	采购成功，资金变为 0
B06	资源文件缺失	删除部分音效/图片	输出警告，主流程不崩溃
B07	存档不存在	无存档文件启动	进入新游戏或提示读取失败

通过边界测试可以看出，本项目在常见异常输入方面进行了必要处理。例如，采购数量为 0、负数或超大时，程序不应继续执行正常采购逻辑；商品售价为负数时，应避免非法数据影响后续结算；当前资金刚好等于采购总价时，应按照正常采购处理，保证资金判断边界正确。

在资源异常方面，图片、音效等资源可能因为路径错误、打包遗漏或文件缺失而无法正常加载。因此项目中采用了资源加载容错思路，即资源缺失时输出警告或使用默认显示，尽量不让程序因为单个资源文件缺失而崩溃。在存档异常方面，如果存档文件不存在或读取失败，程序也应避免直接崩溃，并保持可继续运行的状态。

需要说明的是，本项目没有实现复杂自动化测试框架，上述测试主要通过手动运行、界面操作和观察程序反馈完成。对于课程大作业而言，该测试方式能够覆盖项目主要运行流程和常见异常情况。

4. 测试结果分析

从功能测试结果来看，项目已经能够完成从程序启动到进入下一天的完整主流程。启动页、营业前准备页、营业中页面、每日结算页和店铺升级页之间能够正常跳转，符合项目设计的循环式模拟经营流程。

在核心经营功能方面，商品采购、资金扣除、库存增加、商品价格调整、每日事件显示、营业流程推进和每日结算展示均能够正常执行。资金不足时，程序能够给出提示并避免异常扣费或异常增加库存，说明基本的数据判断逻辑能够正常工作。

在成长与状态保存方面，店铺升级功能可以根据资金情况进行判断和更新；轻量化员工展示能够完成雇佣状态和角色展示，没有扩展为复杂员工管理功能；存档系统可以保存和恢复当前天数、资金、库存、升级状态和员工雇佣状态等关键数据，使游戏进度能够连续保存。

在交互体验方面，统一 UI 音效反馈和鼠标点击特效能够在用户操作时提供反馈，但不影响商品采购、营业结算、升级判断和存档读写等核心经营逻辑。Release 打包版本能够正常运行，说明项目具备基本的提交和展示条件。

综上，本项目已通过主要功能测试、边界测试和运行测试，能够满足课程大作业中“使用 C++ 完成一个图形化小程序”的基本要求。

六、版本迭代与项目管理

本项目使用 Git 和 GitHub 进行版本管理。开发过程中，通过多次 commit 记录项目从基础框架搭建、核心流程实现、界面与交互优化，到最终 Release 打包和展示准备的迭代过程。项目当前主要版本保存在 main 分支中，作为课程大作业提交和后续展示的主要代码版本。

在项目管理过程中，项目目录按照 Data、Logic、Interaction、assets、config 等部分进行组织。其中，Data 用于保存数据相关内容，Logic 用于组织经营逻辑和流程控制，Interaction 用于实现 Qt 页面和用户交互，assets 用于保存图片、音效、字体等资源，config 用于保存配置相关内容。这样的目录划分有助于提高项目结构的清晰度，也便于后续维护和版本迭代。

项目迭代过程大致可以分为以下几个阶段：

表：项目版本迭代记录

版本阶段	迭代主题	主要内容
v0.1	基础框架	建立项目目录、CMake、主要页面框架和资源目录
v0.5	核心流程	完成营业前准备、营业中、每日结算、店铺升级、进入下一天
v0.8	交互优化	接入像素风素材、UI 音效、鼠标点击特效和页面布局优化
v1.0	最终版本	完成存档系统、Release 打包、README、报告与视频展示准备

在版本管理方面，GitHub 不仅用于保存代码，也用于记录开发过程。通过阶段性提交，可以较清楚地看到项目从早期框架到完整流程的变化。与一次性提交全部代码相比，多次 commit 更有利于回溯问题、比较修改内容，也便于展示项目的实际开发过程。

为了提高仓库规范性和项目展示效果，项目中还整理了 README、.gitignore、版本日志、报告文档和 Release 打包版本等内容。其中，README 用于说明项目基本信息、运行方式和功能简介；.gitignore 用于排除不必要的临时文件和构建文件；版本日志用于记录不同阶段的功能变化；报告文档用于课程提交；Release 打包版本用于展示程序最终运行效果。

通过 Git 和 GitHub 的使用，本项目不只是完成了一个本地可运行程序，也形成了相对完整的项目管理记录。这对于课程大作业的提交、后续修改和公开展示都有一定帮助。

七、AI 工具使用说明

在本项目开发过程中，使用了 ChatGPT、Codex 等 AI 工具进行辅助。AI 工具主要用于项目规划、代码修改建议、问题排查、素材提示词设计和文档整理等环节。需要说明的是，AI 工具在项目中起到的是辅助作用，项目主题确定、功能取舍、代码整合、运行测试、素材筛选和最终提交均由本人完成。

在使用 AI 工具时，本项目尽量遵循以下原则：

第一，AI 生成的内容不能直接等同于最终结果。对于代码建议，需要在本地环境中进行编译和运行验证；对于文档内容，需要根据实际完成情况进行修改，避免出现未实现功能被写入报告的情况。

第二，AI 工具可以帮助提高开发效率，但不能替代对项目结构和代码逻辑的理解。项目中的页面跳转、资金库存更新、商品采购、价格调整、每日结算、店铺升级、存档读写等内容，都需要结合实际程序进行测试和确认。

第三，在实验报告中对 AI 工具的使用进行说明，是为了保证课程作业的规范性和透明性。本项目的核心目标、功能取舍、最终运行效果和提交版本均由本人确认，AI 工具主要承担辅助分析、辅助生成建议和辅助整理材料的作用。

八、收获

通过本次高级语言程序设计大作业，我对 C++ 项目开发有了比平时课堂练习更完整的认识。以前学习 C++ 时，更多是在控制台中完成单个题目或小规模程序，重点通常是语法、函数和算法本身。而在《潮汐小镇》这个项目中，程序不再只是解决一道题，而是需要同时考虑数据保存、流程推进、页面交互、资源加载、错误处理和版本管理等多个方面。这让我更直观地理解了类与对象、封装和模块化设计的作用。

在 C++ 程序设计方面，我进一步体会到，合理规划分类和模块可以明显降低项目复杂度。例如，GameManager 负责整体流程协调，Store 保存店铺状态，Product 保存商品数据，Employee 保存轻量化员工状态，页面类负责显示和交互。这样设计之后，不同部分的职责更加清楚，后续修改某个功能时也更容易定位问题。相比把所有逻辑都写在一个页面文件中，模块化结构更适合稍大一点的程序。

在 Qt6 / Qt Widgets 学习方面，我对图形化程序的运行方式有了更具体的理解。Qt 程序不是简单地从上到下执行，而是依赖窗口、控件、按钮事件和信号槽机制进行交互。通过本项目，我理解了按钮点击如何触发槽函数，页面之间如何切换，界面数据如何在用户操作后刷新。这和普通控制台程序有很大不同，也让我认识到图形化程序需要同时关注逻辑正确性和用户操作反馈。

在资源管理方面，本项目使用了图片、音效、字体、鼠标点击特效等资源。开发过程中我认识到，资源路径、文件命名和打包目录非常重要。如果资源缺失或路径错误，程序可能无法

显示预期效果。因此，项目中需要考虑资源加载容错，例如图片或音效缺失时只输出警告，而不是让程序直接崩溃。完成 Release 打包后，我也更加理解了“程序能在开发环境中运行”和“程序能作为打包版本运行”之间的区别。开发环境中能够正常运行，并不代表打包后一定能够正常读取资源。Release 版本需要同时考虑可执行文件、动态库、assets 目录和配置文件之间的相对位置，这让我对图形化项目的发布流程有了更实际的认识。

在页面刷新和数据同步方面，我也遇到了比控制台程序更具体的问题。例如，商品采购后不仅要修改内部数据，还要及时刷新页面上的资金、库存和商品信息；存档读取后，也不能只恢复数据，还要让界面显示与恢复后的状态一致。这个过程让我认识到，图形化程序中的“数据正确”和“界面正确”是两个都需要关注的问题。如果只修改数据而不刷新界面，用户看到的内容就会和程序内部状态不一致；如果只刷新界面但没有真正修改数据，后续结算和存档又会出现问题。

在存档读写方面，我进一步理解了状态保存的重要性。存档并不是简单地写入一个数字，而是要保存能够恢复游戏进度的关键状态，包括当前天数、资金、商品库存、商品售价、升级状态和员工雇佣状态等。读取存档时，也要考虑存档文件不存在、路径错误或内容不完整等情况，避免程序直接崩溃。这个过程让我认识到，程序的稳定性不仅体现在正常流程能运行，也体现在异常情况下能够尽量保持可控。

在项目管理方面，我学会了使用 Git 和 GitHub 记录开发过程。通过多次 commit，可以把项目从基础框架、核心流程、交互优化到最终展示版本的变化保存下来。小步提交比一次性提交更清楚，也更方便回退和排查问题。尤其是在进行页面布局、资源接入和存档功能修改时，如果一次改动过多，出现问题后很难判断具体原因；而小步提交可以把问题限制在较小范围内。README、版本日志、.gitignore 和 Release 文件的整理，也让我认识到一个项目不仅要能运行，还需要有清晰的说明和规范的仓库结构。

总体来说，本次大作业让我经历了从项目构想、功能拆分、代码实现、界面优化、功能测试、版本管理到报告整理的完整过程。虽然项目规模不算很大，但它已经不是单纯的课堂练习，而是一个具有完整流程的 C++ / Qt 图形化程序。通过这个过程，我对程序设计的理解从“写出能运行的代码”进一步转变为“组织一个结构清晰、功能可维护、能够展示的项目”。